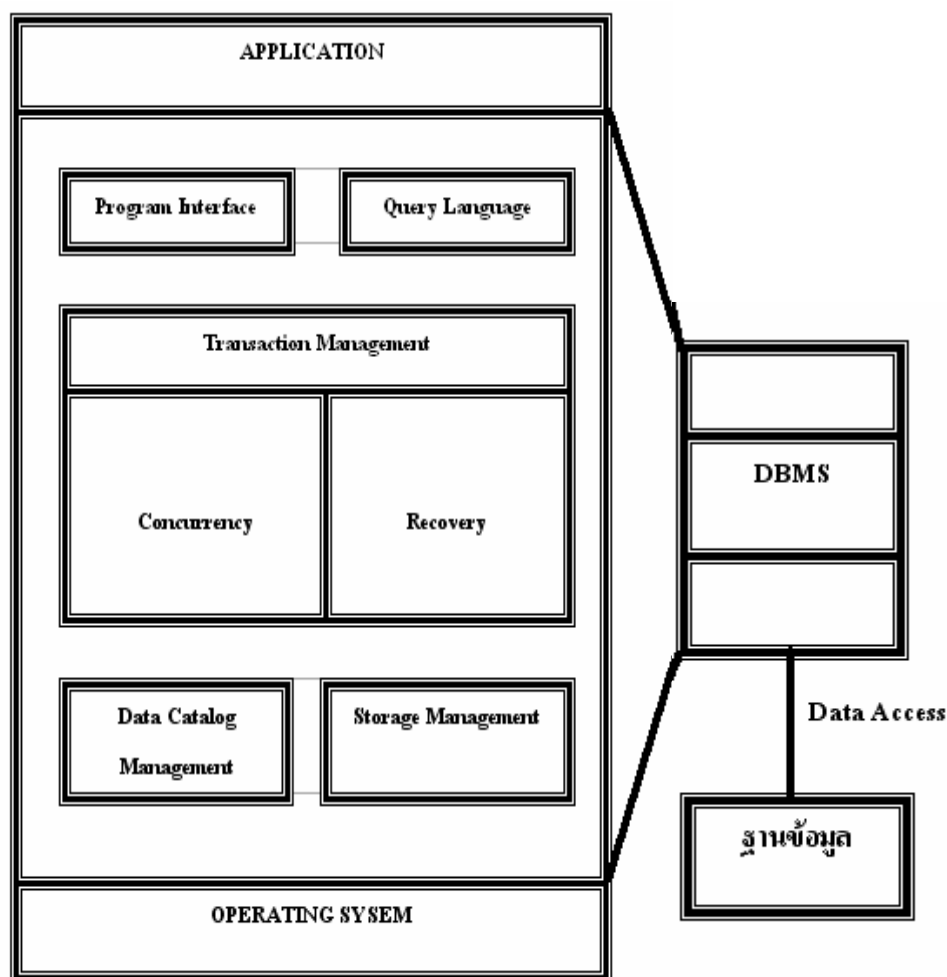


บทที่ 2

ระบบจัดการฐานข้อมูล

ระบบจัดการฐานข้อมูล คือ โปรแกรมหลักในงานฐานข้อมูลที่เป็นตัวกลางระหว่างผู้ใช้กับฐานข้อมูล เพื่อจัดการและควบคุมความถูกต้อง ความซ้ำซ้อน และความสัมพันธ์ของข้อมูลต่างๆ ภายในฐานข้อมูล ทำให้ผู้ใช้และผู้เขียนโปรแกรมสามารถเรียกใช้ข้อมูลจากฐานข้อมูลได้โดยไม่ต้องทราบถึงโครงสร้างทางกายภาพของข้อมูลนั้นๆ ทั้งนี้ระบบจัดการฐานข้อมูลจะต้องทำงานประสานกับระบบปฏิบัติการ(Operating system)



รูปที่ 2-1 ส่วนประกอบของระบบจัดการฐานข้อมูล

2.1 หน้าที่หลักของระบบจัดการฐานข้อมูล

มีรายละเอียดดังนี้

- การจัดการทรานแซกชัน (Transaction Management)
- การฟื้นฟูสภาพ (Recovery)
- การควบคุมภาวะพร้อมกัน (Concurrency Control)
- การประมวลผลคิวรี (Query Processing)
- การจัดการด้านความปลอดภัยของข้อมูล (Security Management)
- การให้บริการภาษาฐานข้อมูล (Database Language service)
- การให้บริการที่ทนต่อความผิดพลาด (Fault Tolerance)
- การจัดการแคตตาล็อกของข้อมูล (Data Catalog Management)
- การจัดการด้านการจัดเก็บข้อมูล (Storage Management)

2.1.1 การจัดการทรานแซกชัน

2.1.1.1 ความหมายของทรานแซกชัน

ทรานแซกชัน หมายถึง หน่วยของงานในระดับตรรกะ (Logical Unit of Work) ที่กระทำกับฐานข้อมูล ซึ่งประกอบด้วยชุดคำสั่งต่างๆ ที่ได้รับอนุญาตให้มีการละเมิดกฎการควบคุมความถูกต้องภายในได้ แต่เมื่อทรานแซกชันทำงานเสร็จสิ้นแล้ว ฐานข้อมูลจะต้องถูกต้องเสมอตามกฎควบคุมความถูกต้อง

หน่วยของงานในระดับตรรกะ คือ ชุดคำสั่งของภาษา เอสคิวแอลหลายคำสั่งนั่นเองทั้งนี้การจัดการทรานแซกชันเป็นหน้าที่ของระบบจัดการฐานข้อมูล

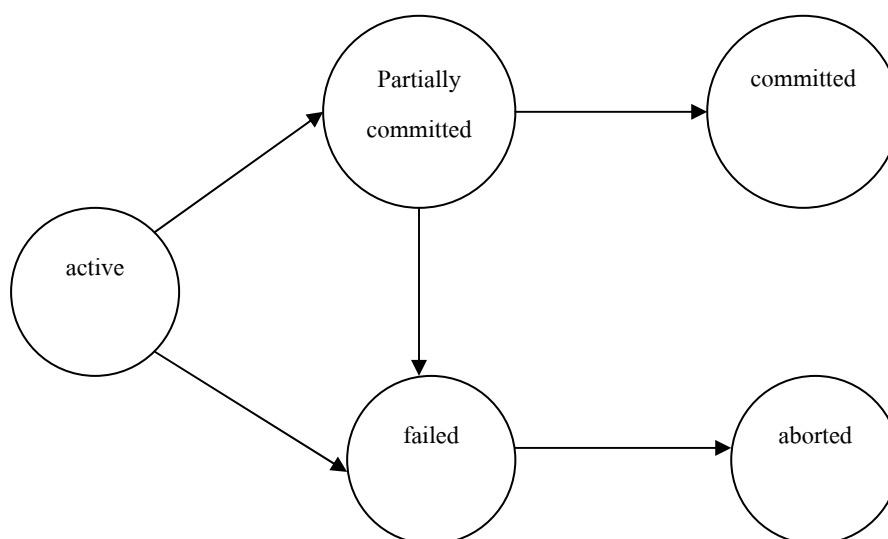
2.1.1.2 คุณสมบัติของทรานแซกชัน

- อะตอมมิกซิตี หมายถึง ทรานแซกชันจะต้องถูกมองเป็นหน่วยเดียวแบ่งย่อยไม่ได้ นั่นคือ ถ้าทรานแซกชันมีทำงานสำเร็จ ก็จะถือว่ามีการทำงานสำเร็จทั้งหมด แต่ถ้าทรานแซกชันผิดพลาด ก็จะถือว่าการทำงานผิดพลาดทั้งหมด ต้องยกเลิกทุกการกระทำในทรานแซกชันนั้น
- คอนซิสเตนซี หมายถึง ทรานแซกชันจะต้องรักษาความถูกต้องนั่นคือ หลังจากกระทำทรานแซกชันเสร็จสิ้นแล้ว ฐานข้อมูลจะต้องถูกต้องเสมอตามกฎควบคุมความถูกต้อง
- ไอโซเลชัน หมายถึง ทรานแซกชัน 2 ทรานแซกชันขึ้นไป เมื่อมีการทำงานไปพร้อมๆ กัน จะต้องไม่มีการรบกวนกัน
- ดูราบิลิตี้ หมายถึง ผลที่เกิดจากการปฏิบัติทรานแซกชันใดๆ จะต้องถาวร

2.1.1.3 สถานะของทรานแซกชัน

ทรานแซกชันใดๆ จะอยู่ในสถานะใดสถานะหนึ่งดังต่อไปนี้ ซึ่งสามารถแบ่งสถานะของทรานแซกชัน ออกเป็น 5 สถานะ คือ

- สถานะเริ่มต้นทำงานของทรานแซกชัน (Active)
- สถานะที่ทรานแซกชันเสร็จสิ้นคำสั่งสุดท้ายแล้วรอการคอมมิต (Partially committed)
- สถานะที่ทรานแซกชันไม่สามารถปฏิบัติงานต่อไปได้ (Fail)
- สถานะที่ทรานแซกชันโรลแบ็ก และฐานข้อมูลกลับคืนไปสู่สถานะก่อนที่ทรานแซกชันจะเริ่มต้นการทำงาน (Abort)
- สถานะที่ทรานแซกชันทำงานเสร็จสมบูรณ์ (Committed)



รูปที่ 2-2 รูปแสดงสถานะของทรานแซกชัน

จากรูปที่ 2-2 เมื่อทรานแซกชันเริ่มต้นทำงาน ทรานแซกชันจะอยู่ในสถานะเริ่มต้นทำงานและเมื่อการกระทำแต่ละคำสั่งของทรานแซกชันเกิดขึ้น สถานะของทรานแซกชันจะถูกเปลี่ยนสถานะจากสถานะเริ่มต้นทำงานเป็นสถานะที่ทรานแซกชันเสร็จสิ้นคำสั่งสุดท้ายแล้วรอคอมมิต เพื่อรอคอมมิตและเมื่อทรานแซกชันคอมมิตแล้ว สถานะของทรานแซกชันจะถูกเปลี่ยนจากสถานะที่ทรานแซกชันเสร็จสิ้นคำสั่งสุดท้ายแล้วรอคอมมิตเป็นสถานะที่ทรานแซกชันทำงานเสร็จสมบูรณ์ แต่ถ้าการกระทำใดการกระทำหนึ่งของทรานแซกชันขณะที่กำลังทำงานหรือขณะที่รอคอมมิต เกิดมีข้อผิดพลาดหรือปัญหาขึ้น สถานะของทรานแซกชันจะถูกเปลี่ยนไปเป็นสถานะที่ทรานแซกชันไม่สามารถปฏิบัติงานต่อไปได้ หลังจากนั้นสถานะของทรานแซกชัน จะถูกเปลี่ยนไปเป็นสถานะที่ทรานแซกชันโรลแบ็ก และฐานข้อมูลกลับคืนไปสู่สถานะก่อนที่ทรานแซกชันจะเริ่มต้นการทำงาน

2.1.2 การฟื้นฟูสภาพ

โดยทั่วไประบบจัดการฐานข้อมูลจะมีระบบรักษาความถูกต้องปลอดภัยของข้อมูล แต่อย่างไรก็ตาม อาจเกิดความขัดข้องหรือความเสียหายของระบบ ที่อาจไปทำลายข้อมูลบางส่วน หรือทำให้ข้อมูลไม่ถูกต้อง ไม่น่าเชื่อถือได้ ดังนั้นจะต้องมีวิธีการที่จะนำข้อมูลที่ถูกลบหายไปกลับคืนมา และอยู่ในสภาพที่ถูกต้อง

2.1.2.1 ประเภทของความขัดข้อง

ประเภทของความขัดข้องจำแนกได้เป็น 3 ลักษณะ

2.1.2.1.1 ความขัดข้องของทรานแซกชัน (Transaction failures)

ความขัดข้องลักษณะนี้จะมีผลกระทบเฉพาะทรานแซกชันที่มีความขัดข้องเกิดขึ้นเท่านั้น ความขัดข้องของทรานแซกชันมี 2 ลักษณะ

- ความขัดข้องที่สามารถป้องกันได้โดยการตรวจสอบภายในโปรแกรม ความขัดข้องดังกล่าวสามารถป้องกันได้ โดยผู้เขียนโปรแกรมต้องตรวจสอบความถูกต้องต่างๆ ภายในโปรแกรมให้ควบคุมทุกกรณีที่มีโอกาสละเมิดกฎควบคุมความถูกต้อง
- ความขัดข้องที่ไม่สามารถป้องกันได้โดยการตรวจสอบภายในโปรแกรม

2.1.2.1.2 ความขัดข้องของระบบ (System failures)

เป็นความขัดข้องที่เกิดผลกระทบต่อทรานแซกชันทั้งหมดที่กำลังดำเนินการอยู่ เช่น ไฟดับ หรือความผิดพลาดของระบบปฏิบัติการ ทำให้ระบบไม่สามารถทำงานต่อไปได้ จำเป็นต้องปิดระบบคอมพิวเตอร์และเปิดเครื่องใหม่อีกครั้งซึ่งจะมีผลต่อทรานแซกชันที่ทำงานอยู่แต่ไม่ทำลายฐานข้อมูล เราเรียกความขัดข้องลักษณะนี้ว่า ซอฟต์แวร์แครช (Soft crash) โดยข้อมูลที่บันทึกในฐานข้อมูลจะไม่สูญหาย แต่ข้อมูลที่อยู่ในหน่วยความจำหลักซึ่งอยู่ระหว่างการทำงานและยังไม่ได้บันทึกจะสูญหายทำให้เราไม่ทราบสถานะที่แท้จริงของทรานแซกชันที่กำลังดำเนินการอยู่ขณะเกิดความขัดข้องได้

2.1.2.1.3 ความขัดข้องของสื่อข้อมูล (Media failures)

เป็นความขัดข้องที่จะเกิดผลกระทบต่อทรานแซกชันทั้งหมดความขัดข้องของสื่อข้อมูลจะมีผลทำลายฐานข้อมูล มีผลกระทบต่อทรานแซกชันที่กำลังใช้งานฐานข้อมูลนั้นอยู่ เราเรียกความขัดข้องลักษณะนี้ว่าฮาร์ดแวร์แครช (Hard crash)

2.1.2.2 หลักการของการฟื้นฟูสภาพ

การแก้ปัญหาความขัดข้องต่างๆ จะใช้ ไฟล์ประวัติ (Log file) เข้ามาช่วยในการฟื้นฟูสภาพ ซึ่งเราเรียกวิธีการนี้ว่า การฟื้นฟูสภาพแบบล็อกเบส (log-base recovery) มีหลักการ คือ การปฏิบัติการของทราน

แซกซ์ชันเกี่ยวกับการบันทึก (write operation) จะต้องบันทึกไว้ในไฟล์ประวัติเสมอ เพื่อบอกถึงสถานะของข้อมูลและสถานะของทรานแซกซ์ชัน

2.1.2.2.1 ไฟล์ประวัติ

รายละเอียดของไฟล์ประวัติ ประกอบด้วย

- ชื่อทรานแซกซ์ชัน คือ ชื่อของทรานแซกซ์ชันที่กระทำการบันทึก
- ชื่อดาต้าไอเทม (Data item name) คือ ชื่อของทรานแซกซ์ชันที่ถูกบันทึก
- ค่าเก่า (Old value) คือ ค่าเก่าของดาต้าไอเทมก่อนการบันทึก
- ค่าใหม่ (New value) คือ ค่าใหม่ของดาต้าไอเทมหลังการบันทึก

2.1.2.2.2 วิธีการฟื้นฟูสภาพ

เนื่องจากการฟื้นฟูสภาพแบบบล็อกเบส จะต้องปฏิบัติตาม กฎเกณฑ์การบันทึกนำหน้า (Write-ahead protocol) ซึ่งกล่าวว่า “เรคคอร์ดประวัติจะต้องบันทึกลงไฟล์ประวัติก่อนเสมอ จึงจะยอมให้มีการบันทึกข้อมูลจากที่พักข้อมูลชั่วคราวหรือบัฟเฟอร์ ลงฐานข้อมูลได้” ด้วยกฎเกณฑ์นี้ทำให้เราสามารถที่จะยกเลิกหรือทำซ้ำทรานแซกซ์ชันใดๆ ที่เกิดขึ้นกับฐานข้อมูลได้โดยใช้ค่าเก่า หรือค่าใหม่ที่เก็บไว้ในไฟล์ประวัติ

การฟื้นฟูสภาพมี 3 วิธี ดังนี้ การยกเลิก (undo) การทำซ้ำ (redo) และจุดตรวจสอบ (checkpoint)

- การยกเลิก คือ การยกเลิกสิ่งที่ทำมาแล้วสำหรับทรานแซกซ์ชันที่กำลังทำงานอยู่แล้ว เกิดความขัดข้องขึ้นระหว่างการดำเนินงานก่อนที่จะคอมมิต ลักษณะนี้จะยกเลิกการกระทำที่ทำมาแล้วทั้งหมด
- การทำซ้ำ คือ การทำซ้ำสำหรับทรานแซกซ์ชันที่ทำงานเสร็จสมบูรณ์แล้วแต่ยังไม่มี การเคลื่อนย้ายข้อมูลจากที่พักข้อมูลชั่วคราว หรือบัฟเฟอร์ ในหน่วยความจำหลักสู่ฐานข้อมูลจริง แล้วเกิดปัญหาขัดข้องซึ่งเสมือนว่าทรานแซกซ์ชันนั้นยังไม่ทำงาน ดังนั้นจึงต้องกระทำซ้ำ
- จุดตรวจสอบ คือ เวลา ณ ขณะใดขณะหนึ่งที่ระบบปฏิบัติการและระบบจัดการฐานข้อมูล ใช้ทำการเคลื่อนย้ายข้อมูลจากที่พักข้อมูลชั่วคราวในหน่วยความจำหลักสู่ฐานข้อมูลในดิสก์โดยจุดตรวจสอบจะถูกบันทึกในไฟล์ประวัติ เป็นระยะๆ เพื่อประโยชน์ในการฟื้นฟูสภาพ การทำจุดตรวจสอบบ่อยๆ ย่อมทำให้การฟื้นฟูสภาพเร็ว แต่อาจจะลดประสิทธิภาพการทำงานของระบบเพราะต้องติดต่อกับส่วนรับเข้า/ส่งออก (input/output)

2.1.3 การควบคุมภาวะพร้อมกัน

หน้าที่ที่สำคัญของระบบจัดการฐานข้อมูลก็คือ การเป็นผู้จัดการให้ผู้ใช้สามารถใช้งานฐานข้อมูลร่วมกันได้อย่างราบรื่นและมีผลลัพธ์ที่ถูกต้อง ซึ่งก็คือการควบคุมภาวะความพร้อมกัน

2.1.3.1 ภาวะพร้อมกัน

คำว่า "ภาวะความพร้อมกัน" หมายความว่า การที่มีทรานแซกชันหลายๆ ทรานแซกชันต้องการเรียกใช้ข้อมูลเดียวกันในเวลาเดียวกัน จากฐานข้อมูลเพื่อทำงานของแต่ละทรานแซกชัน

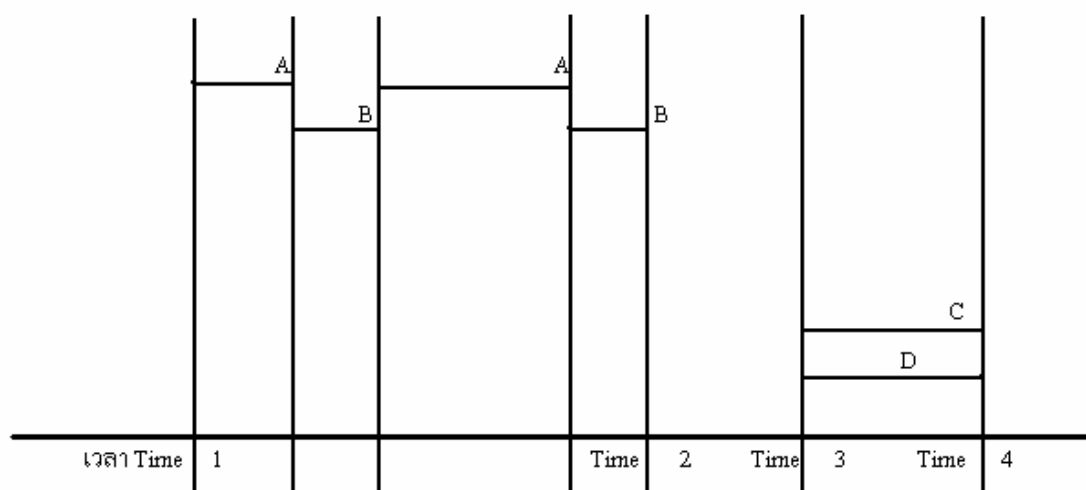
ภาวะพร้อมกันเกิดจากการทำงานได้ 2 ระบบ

- การทำงานในระบบมัลติโปรแกรมมิ่ง (Multiprogramming)
- การทำงานในระบบการประมวลผลในเวลาเดียวกัน (Simultaneous processing)

2.1.3.1.1 การทำงานในระบบมัลติโปรแกรมมิ่ง

การทำงานในระบบมัลติโปรแกรมมิ่งเป็นการทำงานของระบบคอมพิวเตอร์ที่ออกแบบเพื่อให้หน่วยประมวลผลกลาง (Central processing unit) ทำงานในหลายๆงานในขณะเดียวกันได้ หากไม่มีการทำงานแบบมัลติโปรแกรมมิ่ง ซีพียูต้องทำงานงานหนึ่งงานใดจนเสร็จ จึงสามารถทำงานอื่นต่อได้ ซึ่งหมายความว่าขณะที่งานนั้นกำลังใช้อุปกรณ์อื่นที่ไม่ใช่ซีพียู ซีพียูต้องเสียเวลารอ ก็คือไม่ได้ถูกใช้งาน ดังนั้นจึงเกิดแนวความคิดการประมวลผลแบบให้หลายโปรแกรมทำงานพร้อมกัน โดยมีการสลับช่วงการทำงานระหว่างโปรแกรม เพื่อสลับให้ซีพียูไปทำงานของทรานแซกชันอื่นๆ ซึ่งแต่ละทรานแซกชันที่ให้ซีพียูทำงานอาจเป็นของโปรแกรมเดียวกันหรือต่างโปรแกรมก็ได้ โดยใช้หลักการอินเทอร์ลีฟ (interleaved)

อินเทอร์ลีฟ คือการที่ทรานแซกชันมากกว่าหนึ่งทรานแซกชัน มีการสลับการทำงานกัน ในขณะใดขณะหนึ่ง โดยที่ระบบจัดการฐานข้อมูลจะต้องควบคุมภาวะการพร้อมกัน เพื่อให้แต่ละทรานแซกชันมีการทำงานสลับกันไปมา ทั้งนี้ผลลัพธ์ที่ได้จะต้องมีความถูกต้องเสมือนว่าแต่ละทรานแซกชันทำงานตามลำดับเรียงกันที่แต่ละทรานแซกชัน ตัวอย่างดังรูปที่ 2-3 มีทรานแซกชัน A และ B ทำงานพร้อมกันในช่วงเวลา time1 และ time2 โดยระบบจัดการฐานข้อมูลจะต้องควบคุมภาวะความพร้อมกัน โดยมีการสลับการทำงานระหว่างทรานแซกชัน A และ B



รูปที่ 2-3 การทำงานแบบอินเทอร์ลีฟ เทียบกับ การประมวลผลในเวลาเดียวกัน

2.1.3.1.2 การประมวลผลในเวลาเดียวกัน

การประมวลผลในเวลาเดียวกัน เป็นการทำงานในระบบคอมพิวเตอร์ที่มีซีพียูมากกว่าหนึ่งซีพียู เพื่อรองรับการทำงานของโปรแกรมใดโปรแกรมหนึ่ง ได้โดยไม่ต้องสลับทำงานระหว่างทรานแซกชัน ดังนั้นซีพียูตัวใดตัวหนึ่งจะทำงานของโปรแกรมใดโปรแกรมแยกกันไปจนเสร็จงาน ดังรูปที่ 2-3 ทรานแซกชัน C และ D ทำงานในช่วงเวลา time3 ถึง time4 โดยแต่ละทรานแซกชันมีหน่วยประมวลผลแยกกันไปคนละซีพียู

2.1.3.2 ความจำเป็นในการควบคุมภาวะพร้อมกัน

การควบคุมภาวะภาวะพร้อมกันในการใช้งานฐานข้อมูลเป็นสิ่งสำคัญอย่างยิ่ง เพราะหากระบบจัดการฐานข้อมูลไม่มีกลไกดังกล่าวย่อมจะก่อให้เกิดปัญหาในการทำงาน ดังนี้

- ปัญหาการสูญหายของข้อมูลที่มีการปรับปรุงแก้ไข (The lost update problem) เป็นปัญหาที่เกิดจากทรานแซกชันมากกว่าหนึ่งทรานแซกชัน ต้องการปรับปรุงข้อมูลเดียวกันในเวลาไล่เลี่ยกันทำให้ผลลัพธ์ที่ได้ไม่ถูกต้องเพราะข้อมูลที่ถูกแก้ไขโดยทรานแซกชันก่อนหน้าหายไปหมด จะปรากฏแค่ผลลัพธ์ที่เกิดจากการแก้ไขของทรานแซกชันหลังสุดเท่านั้น
- ปัญหาจากการเรียกใช้ข้อมูลชุดเดียวกันของทรานแซกชันที่ยังไม่คอมมิต (Uncommitted dependency) เป็นปัญหาที่เกิดจากทรานแซกชันมากกว่าหนึ่งทรานแซกชันต้องการเรียกใช้ข้อมูลชุดเดียวกัน โดยทรานแซกชันที่ 1 ยังอยู่ระหว่างกลางในการทำงาน ขณะเดียวกันทรานแซกชันที่ 2 เรียกใช้ข้อมูลที่ถูกแก้ไขโดยทรานแซกชันที่ 1 หลังจากนั้นปรากฏว่าทรานแซกชันที่ 1 มีปัญหา จะต้องถูกยกเลิกและโรลแบ็กเพื่อเริ่มต้นการทำงานใหม่ทั้งหมดดังนั้นข้อมูลที่ทรานแซกชันที่ 2 เรียกเข้าไปใช้งานแล้วจึงเป็นข้อมูลที่ไม่ถูกต้อง ทำให้ผลลัพธ์ที่ไม่ถูกต้อง
- ปัญหาการเรียกใช้ข้อมูลที่ไม่สอดคล้องกัน (Inconsistent retrieval problem) เป็นปัญหาที่เกิดจากทรานแซกชันมากกว่าหนึ่งทรานแซกชัน มีการใช้ข้อมูลชุดเดียวกัน โดยทรานแซกชันหนึ่งใช้ข้อมูลนั้นเพื่อประมวลผลใดๆ ในขณะที่เดียวกันก็มีทรานแซกชันอื่นได้มีการปรับปรุงแก้ไขข้อมูลชุดเดียวกัน ทำให้ผลลัพธ์จากทรานแซกชันแรกไม่ถูกต้อง

2.1.3.3 วิธีการควบคุมภาวะพร้อมกัน

ในการควบคุมภาวะพร้อมกัน ระบบจัดการฐานข้อมูลจะต้องมีกลไกสำหรับจัดเรียงลำดับก่อนหลังการทำงานของแต่ละทรานแซกชัน ซึ่งกลไกดังกล่าวอาจใช้วิธีใดวิธีหนึ่งต่อไปนี้ในการจัดเรียงลำดับทรานแซกชัน ได้แก่ ล็อกกิง, ไทม์สเตมป์ และออฟทิมิสติก

2.1.3.3.1 ล็อกกิง

เป็นวิธีการกำหนดสถานะของข้อมูลเพื่อให้แต่ละทรานแซกชันที่ต้องการจะใช้ข้อมูล ได้ล็อกข้อมูลนั้นในฐานข้อมูลไม่ให้ทรานแซกชันอื่นใช้งานได้จนกว่าทรานแซกชันที่ใช้ข้อมูลนั้นจะทำงานเสร็จสิ้นสมบูรณ์ก่อนแล้วจึงคลายล็อกข้อมูลนั้น ทรานแซกชันอื่นจึงสามารถเข้าไปใช้ข้อมูลชุดนั้นๆ ได้

การล็อกแบ่งออกเป็น 2 ประเภท

- **เอ็กคลูซีฟล็อก** คือการกำหนดสถานะของข้อมูลให้เป็นล็อก โดยไม่ให้ ทรานแซกชันอื่น ใช้ข้อมูลที่ถูกล็อกนั้น การกำหนดให้ข้อมูลนั้นเป็นเอ็กคลูซีฟล็อก ได้จะเป็นกรณีที่ทรานแซกชันนั้นต้องการเปลี่ยนแปลงแก้ไขหรือบันทึกข้อมูล
- **แชร์ล็อก** คือการกำหนดสถานะข้อมูลให้ใช้งานร่วมกันกับทรานแซกชันอื่นได้

โดยการล็อกข้อมูลในฐานข้อมูลจะมีการแบ่งกลุ่มหรือระดับการล็อกได้หลายระดับเพื่อให้เกิดความยืดหยุ่นและความคล่องตัวในการใช้งาน โดยจะมีระดับในการล็อกดังนี้

- การล็อกฐานข้อมูลทั้งฐาน เป็นการล็อกทั้งฐานข้อมูลในขณะที่ทรานแซกชันใดทรานแซกชันหนึ่งกำลังใช้งาน
- การล็อกบางส่วนของฐานข้อมูล
- การล็อกเฉพาะตารางใดตารางหนึ่งในฐานข้อมูล
- การล็อกเฉพาะแถวบางแถวในฐานข้อมูล
- การออกเฉพาะบางฟิลด์หรือบางแอตทริบิวต์ในฐานข้อมูล

แต่วิธีการนี้ก็ยังมีข้อเสียอยู่เหมือนกันก็คือ การเกิดเดดล็อก (deadlock) นั่นคือ เหตุการณ์ที่ทรานแซกชันรอการใช้ข้อมูลที่ถูกล็อก โดยทรานแซกชันอื่นอย่างไม่รู้จบในลักษณะลูกโซ่ เช่น การที่ ทรานแซกชันที่ 2 รอคอยเพื่อใช้ทรัพยากร ที่กำลังถูกล็อกไว้แล้วใช้งานโดยทรานแซกชันที่ 1 ดังนั้นทรานแซกชันที่ 2 จะต้องรอจนกว่าทรานแซกชันที่ 1 จะทำงานเสร็จและปลดล็อกทรัพยากรนั้น แต่ในขณะเดียวกัน ทรานแซกชันที่ 1 ก็รอใช้งานทรัพยากรที่กำลังถูกใช้งานและถูกล็อกโดยทรานแซกชันที่ 2 เช่นเดียวกัน ดังนั้นจึงไม่มีทรานแซกชันใดสามารถดำเนินการต่อไปได้ จะต้องอยู่ในสภาพหยุดนิ่งและรอไปเรื่อยๆ ไม่รู้จบ โดยวิธีการแก้ปัญหาเดดล็อก มีดังนี้

- **การป้องกันก่อนเกิดเดดล็อก (Deadlock prevention)** วิธีนี้ระบบจัดการฐานข้อมูลในส่วนของการควบคุมภาวะพร้อมกันจะกำหนดว่า ทรานแซกชันที่ต้องการเรียกใช้ข้อมูลใดก็ตามจะต้องล็อกข้อมูลทุกอันที่ต้องการเรียกใช้ทั้งหมดไว้ก่อนการใช้งาน ถ้าหากทรานแซกชันไม่สามารถจะล็อกข้อมูลไว้ล่วงหน้าได้ก็จะต้องรอจนกว่าจะล็อกได้ครบเสียก่อนจึงเริ่มต้นทำงานได้
- **การตรวจจับการเกิดเดดล็อก (Deadlock detection)** วิธีนี้เป็นวิธีการแก้ปัญหาเมื่อเกิดเดดล็อกแล้ว โดยระบบจัดการฐานข้อมูลจะตรวจจับว่าทรานแซกชันใดทำให้เกิดเดดล็อกบ้าง ซึ่งเมื่อตรวจเจอระบบจัดการฐานข้อมูลจะทำการตรวจสอบและยกเลิก

ทรานแซกชันทำให้เกิดเดดล็อก และวิธีนี้จะสิ้นเปลืองมาก เนื่องจากระบบจัดการฐานข้อมูลจะต้องทำการตรวจสอบการเกิดเดดล็อกอยู่ตลอดเวลา

2.1.3.3.2 วิธีโทรม์แสดมปี

โทรม์แสดมปี คือ ค่าที่ระบบจัดการฐานข้อมูลกำหนดขึ้น เพื่อระบุลำดับของแต่ละทรานแซกชันในการเข้าทำงาน ดังนั้นวิธีนี้จะไม่ทำให้เกิดปัญหาเดดล็อกขึ้น

2.1.3.3.3 วิธีออบทิมิสติก

มีแนวคิดทำให้ทรานแซกชันทำงานใช้ข้อมูลตามปกติ ข้อมูลที่มีการแก้ไขหรือเปลี่ยนแปลงโดยแต่ละทรานแซกชันจะไม่บันทึกในฐานข้อมูลโดยตรง แต่จะบันทึกไว้ในพื้นที่อื่นๆ ที่กำหนดไว้ชั่วคราวของแต่ละทรานแซกชันจนกว่าทรานแซกชันนั้นจะทำงานเสร็จสิ้นสมบูรณ์ก่อน ทั้งนี้ก่อนนำข้อมูลที่เก็บไว้ในพื้นที่ชั่วคราวมาบันทึกถาวรในฐานข้อมูล ระบบจัดการฐานข้อมูลจะต้องตรวจสอบเสียก่อนว่าทรานแซกชันนั้นมีการทำงานแบบขั้นตอนไม่เป็นลำดับ ซึ่งจะทำให้เกิดข้อมูลขัดแย้งกับข้อมูลของทรานแซกชันอื่นหรือไม่ หากตรวจแล้วไม่มีความขัดแย้งก็จะนำข้อมูลที่เก็บไว้ในพื้นที่ชั่วคราวบันทึกลงในฐานข้อมูลอย่างถาวรต่อไป แต่ถ้าหากตรวจสอบแล้วมีปัญหาระบบจัดการฐานข้อมูลก็จะยกเลิกผลลัพธ์ของทรานแซกชันนั้น

2.1.4 การประมวลผลคิวรี

2.1.4.1 ความหมายของการประมวลผลคิวรี

การประมวลผลคิวรี เป็นหน้าที่หนึ่งของระบบจัดการฐานข้อมูลโดยระบบจัดการฐานข้อมูลจะทำการประมวลผลคิวรีที่เกิดจากภาษาคิวรี(query language) และหาคำตอบให้ผู้ใช้ได้อย่างเหมาะสมและรวดเร็ว

การประมวลผลคิวรี เป็นกระบวนการหนึ่งของระบบจัดการฐานข้อมูลที่ใช้ในการเลือกแผนหรือกลยุทธ์ที่เหมาะสมในการคิวรีข้อมูลจากฐานข้อมูลโดยระบบจัดการฐานข้อมูลจะมีตัวประมวลผลที่เรียกว่า ตัวประมวลผลคิวรี ทำหน้าที่ในการเลือกแผนหรือกลยุทธ์ที่เหมาะสม

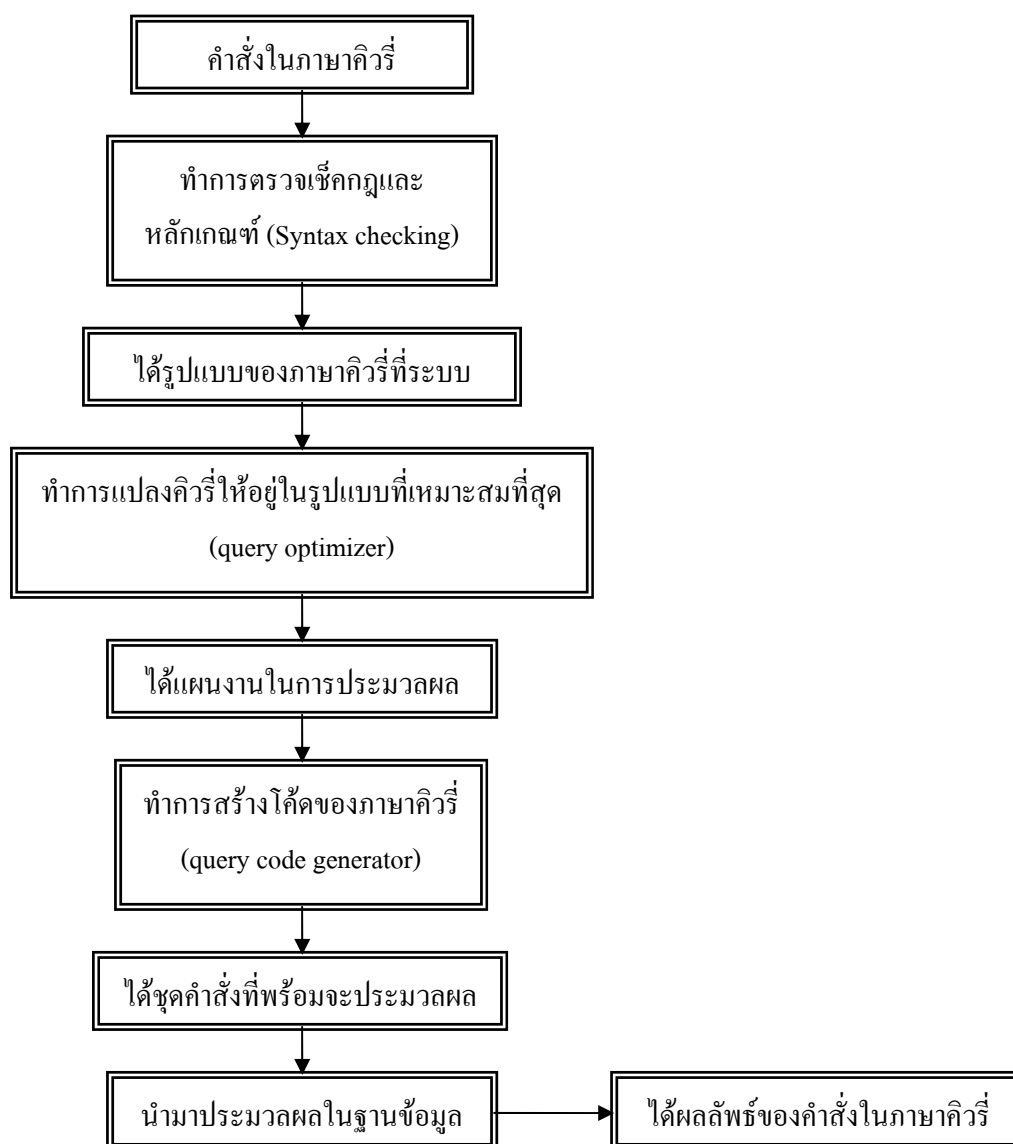
2.1.4.2 ขั้นตอนของการประมวลผลในภาษาคิวรี

การประมวลผลคิวรีจะทำการประมวลเป็นขั้นตอนต่างๆ โดยจะนำคำสั่งภาษาคิวรีที่ผู้ใช้ต้องการมาตรวจสอบกับกฎและไวยากรณ์ (rule and grammars) ของภาษาคิวรีนั้นๆ ระบบจัดการฐานข้อมูลจะใช้กลยุทธ์ในการประมวลผล สำหรับหาผลลัพธ์ของคำสั่งภาษาคิวรีนั้นๆ จะทำให้ได้ผลลัพธ์ในเวลารวดเร็ว

จากรูปที่ 2-4 ขั้นตอนที่ 1 การตรวจเช็คกฎไวยากรณ์ของภาษา เมื่อผู้ใช้ได้ทำการป้อนคำสั่งภาษาคิวรี ระบบจัดการฐานข้อมูลโดยตัวประมวลผลคิวรี จะทำการตรวจเช็คกฎและหลักเกณฑ์ ของภาษาคิวรีนั้น เมื่อตรวจเช็คแล้วหากพบว่าคำสั่งภาษาคิวรีที่ป้อนเข้ามาในระบบมีความผิดพลาดไม่เป็นไปตามกฎเกณฑ์ของภาษาคิวรี ระบบจัดการฐานข้อมูลโดยตัวประมวลผลคิวรีก็จะส่งข้อความไปยังผู้ใช้ ให้ผู้ใช้

ปรับแก้ให้ถูกต้อง แต่ถ้าตรวจเช็คแล้วพบว่าคำสั่งภาษาคิวรีที่ป้อนเข้ามาในระบบไม่มีความผิดพลาด เป็นไปตามกฎเกณฑ์ของภาษาคิวรี ก็จะทำการแปลงให้ได้รูปแบบของภาษาคิวรี เพื่อให้ระบบจัดการฐานข้อมูลสามารถเข้าใจได้ ซึ่งผลการแปลงจะอยู่ในรูปพีชคณิตแบบสัมพันธ์ เพื่อให้ระบบจัดการฐานข้อมูลเข้าใจและทำตามผลที่ได้นั้น

ขั้นตอนที่ 2 การแปลงคิวรีให้อยู่ในรูปแบบที่เหมาะสมที่สุด เมื่อได้สมการพีชคณิตแบบสัมพันธ์แล้ว ระบบจัดการฐานข้อมูลจะทำการประมวลผล สมการพีชคณิตแบบสัมพันธ์นั้นเพื่อให้ได้แผนการเข้าถึงข้อมูลในการประมวลผล ในการเข้าถึงข้อมูลนั้นจะมีวิธีการเข้าถึงฐานข้อมูลได้หลายวิธีระบบจัดการฐานข้อมูลจะทำการหาวิธีที่เหมาะสมในการเข้าถึงฐานข้อมูลโดยจะมีเทคนิคต่างๆ ในการเข้าถึงข้อมูลที่ใช้เวลาน้อยที่สุด หรือเรียกว่าการทำออปติไมเซชัน



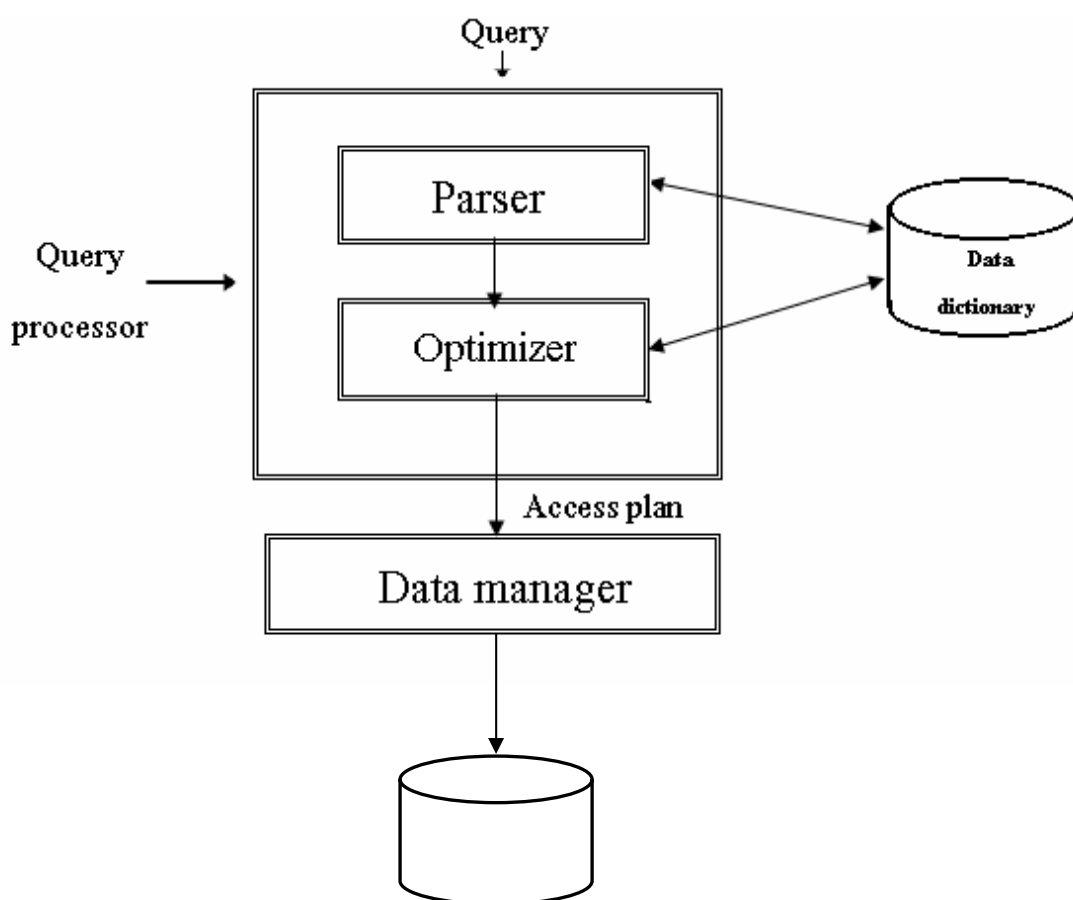
รูปที่ 2-4 ขั้นตอนการประมวลผลในภาษาคิวรี

ขั้นตอนที่ 3 การสร้างชุดคำสั่งของภาษาคิวรี เมื่อได้วิธีที่ดีที่สุดในการเข้าถึงฐานข้อมูลแล้ว ระบบจัดการฐานข้อมูลโดยตัวประมวลผลคิวรีจะทำการสร้างชุดคำสั่งของภาษาคิวรี ทำให้ได้ชุดคำสั่งที่พร้อมจะประมวลผล

ขั้นตอนที่ 4 การประมวลผลในฐานข้อมูล เมื่อได้ชุดคำสั่งที่พร้อมจะประมวลผลแล้ว ตัวประมวลผลคิวรีก็จะทำการคอมไพล์หรือแปลคำสั่ง และ ประมวลผลชุดคำสั่งจนได้ผลลัพธ์

2.1.4.3 โครงสร้างของตัวประมวลผลคิวรี

ตัวประมวลผลคิวรีมีโครงสร้างดังรูปที่ 2-5 มีส่วนประกอบดังนี้



รูปที่ 2-5 โครงสร้างของตัวประมวลผลคิวรี

2.1.4.3.1 ส่วนตรวจสอบภาษา (Parser)

เป็นส่วนหนึ่งของตัวประมวลผลคิวรีที่ทำหน้าที่ตรวจสอบเช็คกฎและหลักเกณฑ์ ของคำสั่งค้นหาข้อมูลที่กำหนดไว้ในระบบฐานข้อมูล โดยจะทำการตรวจสอบข้อมูลที่อยู่ในพจนานุกรมข้อมูล แล้วทำการแปลคำสั่งให้อยู่ในรูปของสมการพีชคณิตแบบสัมพันธ์ (relational algebraic expression) ที่ระบบจัดการฐานข้อมูลเข้าใจ

2.1.4.3.2 ออปติไมเซอร์

เป็นส่วนหนึ่งของตัวประมวลผลคิวรีที่ทำหน้าที่ในการหากลยุทธ์หรือวิธีที่เหมาะสม เพื่อให้การเข้าถึงข้อมูลประหยัดเวลาที่สุดเมื่อคำสั่งในภาษาคิวรีผ่านตัวตรวจสอบภาษาแล้วจะได้คำสั่งในการค้นหาข้อมูลที่อยู่ในรูปของสมการพีชคณิตแบบสัมพันธ์ ซึ่งสามารถประมวลผลได้หลายวิธีด้วยเทคนิคต่างๆ ในการทำการหาทางเลือกที่เหมาะสม ออปติไมเซอร์จะทำการเลือกการเข้าถึงข้อมูลที่เหมาะสมที่สุดขึ้นมา โดยอาจนำเอาข้อต่อไปนี้เป็นมาใช้ในการพิจารณาเช่น

- การลดปริมาณการดึงหรือเรียกใช้ข้อมูล
- การลดขนาดของโครงสร้างข้อมูลโดยพิจารณาใช้เทคนิคการลดการทำงานระหว่างการดึงข้อมูล
- จัดลำดับคำสั่งเพื่อให้ขั้นตอนการทำงานลดลง เช่น จัดลำดับคำสั่ง SELECT เพื่อลดการนำเข้าข้อมูลตั้งแต่เริ่มต้นดึงข้อมูลจากดิสก์

ฯลฯ